

Projeto Integrador - Simulador SIR (Epidemiológico)

1. Visão Geral do Sistema de Dados

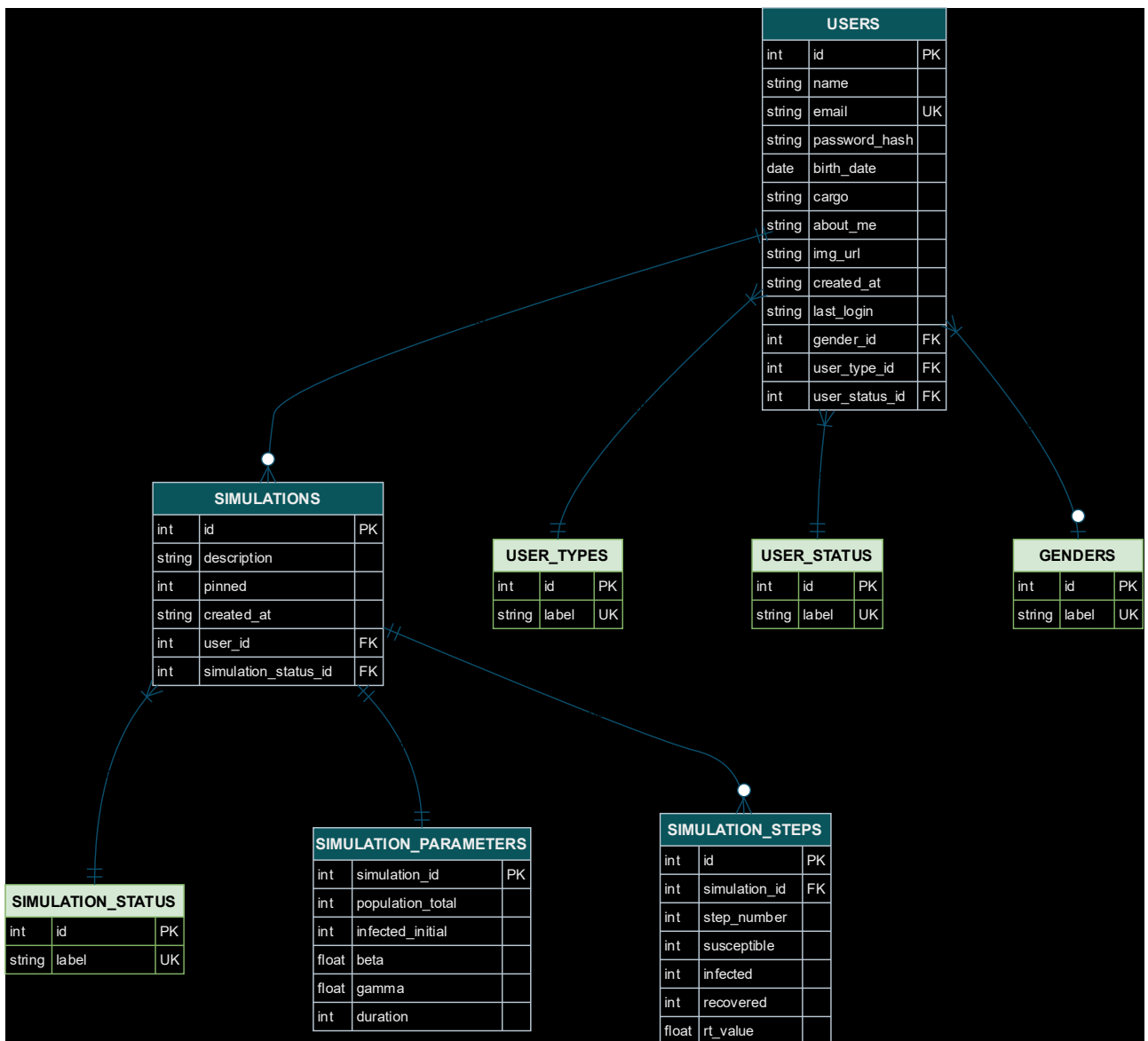
Tecnologias Utilizadas

Componente	Tecnologia
SGDB	SQLite / PostgreSQL
ORM	SQLAlchemy (Flask-SQLAlchemy)
Linguagem	Python 3.x
Framework Web	Flask

Porquê SQLite + ORM?

- **SQLite** (dev): Base de dados leve, baseada em ficheiro (sem necessidade de servidor separado), ideal para desenvolvimento e projetos de escala moderada
- **PostgreSQL** (Produção): SGBD robusto e escalável, usado em produção no **Render** (cloud hosting)
- **ORM (Object-Relational Mapping)**: Abstrai a complexidade SQL, permitindo trabalhar com objetos Python em vez de queries SQL diretas (evitando SQL injection e permitindo mobilidade)

2. Modelo de Dados (Diagrama ER)



3. Arquitectura das Tabelas

3.1 Tabelas de Lookup (Normalização)

Usamos tabelas de lookup para **normalizar a base de dados** e garantir integridade referencial.

Tabela	Propósito	Valores
<code>user_types</code>	Tipos de utilizador	registered, admin, researcher
<code>user_status</code>	Estado da conta	active, pending, suspended
<code>genders</code>	Género do utilizador	Male, Female, Other, Prefer not to say
<code>simulation_status</code>	Estado da simulação	complete, running, paused, failed, deleted

Vantagens da Normalização:

- **Menos redundância** - guardamos IDs em vez de strings repetidas
- **Consistência** - evita erros como "Admin" vs "admin"
- **Integridade referencial** - FK garante que valores são válidos

3.2 Tabelas Principais

Tabela `users` (Utilizadores)

```
class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=True)
    email = db.Column(db.String(100), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False) #hash e ã pw
    birth_date = db.Column(db.Date, nullable=True)
    cargo = db.Column(db.String(20), nullable=True)
    about_me = db.Column(db.Text, nullable=True)
    img_url = db.Column(db.String(255), nullable=True)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    last_login = db.Column(db.DateTime, nullable=True)

    # Foreign Keys (Chaves Estrangeiras)
    gender_id = db.Column(db.Integer, db.ForeignKey('genders.id'))
    user_type_id = db.Column(db.Integer, db.ForeignKey('user_types.id'))
    user_status_id = db.Column(db.Integer, db.ForeignKey('user_status.id'))

    # Relacionamentos
    simulations = db.relationship('Simulation', backref='creator',
    lazy=True)
```

Tabela `simulations` (Simulações)

```
class Simulation(db.Model):
    __tablename__ = 'simulations'

    id = db.Column(db.Integer, primary_key=True)
    description = db.Column(db.Text, nullable=True)
    pinned = db.Column(db.Boolean, default=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    # Foreign Keys
    user_id = db.Column(db.Integer, db.ForeignKey('users.id'),
        nullable=False)
    simulation_status_id = db.Column(db.Integer,
        db.ForeignKey('simulation_status.id'))

    # Relacionamentos com CASCADE DELETE
    parameters = db.relationship('SimulationParameters', uselist=False,
        cascade="all, delete-orphan")
    steps = db.relationship('SimulationSteps', cascade="all, delete-orphan")
```

Tabela `simulation_parameters` (Parâmetros)

```
class SimulationParameters(db.Model):
    __tablename__ = 'simulation_parameters'

    # Chave primária E estrangeira (relação 1:1)
    simulation_id = db.Column(db.Integer, db.ForeignKey('simulations.id'),
        primary_key=True)

    population_total = db.Column(db.BigInteger, nullable=False)
    infected_initial = db.Column(db.BigInteger, nullable=False)
    beta = db.Column(db.Float, nullable=False) # Taxa de transmissão
    gamma = db.Column(db.Float, nullable=False) # Taxa de recuperação
    duration = db.Column(db.Integer, nullable=False)
```

Tabela `simulation_steps` (Resultados Diários)

```
class SimulationSteps(db.Model):
    __tablename__ = 'simulation_steps'

    id = db.Column(db.Integer, primary_key=True)
    simulation_id = db.Column(db.Integer, db.ForeignKey('simulations.id'))
    step_number = db.Column(db.Integer, nullable=False) # Dia

    # Métricas SIR
    susceptible = db.Column(db.BigInteger, nullable=False)
    infected = db.Column(db.BigInteger, nullable=False)
    recovered = db.Column(db.BigInteger, nullable=False)
    rt_value = db.Column(db.Float, nullable=True) # Número Reprodutivo
```

4. Tipos de Relacionamentos

Relação	Tipo	Descrição
User → Simulation	1:N	Um utilizador pode criar várias simulações
Simulation → Parameters	1:1	Cada simulação tem exatamente um conjunto de parâmetros
Simulation → Steps	1:N	Uma simulação gera múltiplos passos (dias)
User → UserType	N:1	Vários utilizadores podem ter o mesmo tipo
User → Genders	N:1	Opcional, através de FK nullable

5. Operações com o ORM (Exemplos de Queries)

5.1 CREATE - Criar Registos

```
# Criar novo utilizador
new_user = User(
    name="João Silva",
    email="joao@email.com",
    password_hash=generate_password_hash("password123"),
    user_type_id=1,
    user_status_id=1
)
db.session.add(new_user)
db.session.commit()
```

SQL Equivalente:

```
INSERT INTO users (name, email, password_hash, user_type_id, user_status_id)
VALUES ('João Silva', 'joao@email.com', 'pbkdf2:sha256:...', 1, 1);
```

5.2 READ - Consultar Dados

```
# Buscar por ID
user = User.query.get(1)

# Buscar por condição (WHERE)
user = User.query.filter_by(email="joao@email.com").first()

# Contar registos (COUNT)
total_users = User.query.count()

# Filtro com JOIN implícito
admin_type = UserType.query.filter_by(label='admin').first()
admins = User.query.filter_by(user_type_id=admin_type.id).all()
```

SQL Equivalente:

```
SELECT * FROM users WHERE id = 1;
SELECT * FROM users WHERE email = 'joao@email.com' LIMIT 1;
```

```
SELECT COUNT(*) FROM users;
SELECT u.* FROM users u
  JOIN user_types t ON u.user_type_id = t.id
 WHERE t.label = 'admin';
```

5.3 UPDATE - Atualizar Dados

```
user = User.query.get(1)
user.name = "João Silva Atualizado"
user.last_login = datetime.utcnow()
db.session.commit()
```

SQL Equivalente:

```
UPDATE users
SET name = 'João Silva Atualizado', last_login = '2025-01-30 10:00:00'
WHERE id = 1;
```

5.4 DELETE - Apagar Registros

```
simulation = Simulation.query.get(5)
db.session.delete(simulation)
db.session.commit()
# CASCADE: apaga automaticamente parameters e steps!
```

SQL Equivalente:

```
DELETE FROM simulation_steps WHERE simulation_id = 5;
DELETE FROM simulation_parameters WHERE simulation_id = 5;
DELETE FROM simulations WHERE id = 5;
```

6. Funcionalidades Chave de BD no Projeto

6.1 Seed Data (Povoamento Inicial)

```
def seed_database():
    """Popula a BD com valores iniciais das lookup tables"""

    # Tipos de utilizador
    types = ['registered', 'admin', 'researcher']
    for t in types:
        if not UserType.query.filter_by(label=t).first():
            db.session.add(UserType(label=t))

    # ... outros Lookups ...
    db.session.commit()
```

6.2 Segurança - Hash de Passwords

```
from werkzeug.security import generate_password_hash, check_password_hash

# Guardar password (nunca em texto claro!)
password_hash = generate_password_hash("user_password")

# Verificar password no login
if check_password_hash(user.password_hash, provided_password):
    # Password correta
```

6.3 Cascade Delete (Integridade referencial)

```
# Definição no modelo
steps = db.relationship('SimulationSteps',
                        cascade="all, delete-orphan")

# Quando apagamos uma simulação, os steps são apagados automaticamente
```

6.4 Backref e Lazy Loading

```
# Relação com Lazy=True
simulations = db.relationship('Simulation', backref='creator', lazy=True)

# As simulações só são carregadas quando acedemos:
user.simulations # <- Query executada aqui!
```

7. Estatísticas da Base de Dados

Queries Agregadas

```
# Dashboard de estatísticas
def get_dashboard_stats():
    total_users = User.query.count()
    total_simulations = Simulation.query.count()

    complete_status =
SimulationStatus.query.filter_by(label='complete').first()
    completed_sims = Simulation.query.filter_by(
        simulation_status_id=complete_status.id
    ).count()

    return {
        'total_users': total_users,
        'total_simulations': total_simulations,
        'completed_simulations': completed_sims
    }
```